

Middle of a Linked List

Personal DSA Study Notes — Java

1. Introduction

The task is to find the middle node of a singly linked list and return the actual node, not just its value. For an odd-length list there is one exact middle. For an even-length list there are two middle positions; the required answer is the second middle node.

These notes cover the two-pass length-based approach and the optimized one-pass slow/fast pointer approach.

2. Problem Statement

Given the head of a linked list, return the node that represents the middle. If the list has an even number of nodes, return the second of the two middle nodes.

Examples:

List	Answer
1 → 2 → 3 → 4 → 5	3
1 → 2 → 3 → 4 → 5 → 6	4 (second middle)

3. Required Concepts

- Linked-list traversal: move through nodes using `current = current.next`.
- Integer division: `count / 2` uses integer division in Java.
- Node reference: the answer must be the `ListNode` object itself.
- Two-pointer technique: slow moves one node at a time while fast moves two.

4. Approach 1 — Find Length, Then Reach the Middle

First count the number of nodes n . The required middle position is $(n / 2) + 1$ using 1-based positions. For $n = 5$, this gives 3. For $n = 6$, integer division gives $3 + 1 = 4$, which is the second middle.

The list is traversed once to count nodes and then again to reach the required position.

Pseudocode

```
count = 0
temp = head

while temp != null:
    count = count + 1
    temp = temp.next

middlePosition = count / 2 + 1

temp = head
while middlePosition > 1:
    temp = temp.next
    middlePosition = middlePosition - 1

return temp
```

Original Core Algorithm

```
temp = head
count = 0
```

```

while temp != null:
    count++
    temp = temp.next

middle = count / 2 + 1
temp = head

while temp != null:
    middle--
    if middle == 0:
        break
    temp = temp.next

return temp

```

Complete Java Code

```

class Solution {
    public ListNode middleOfLinkedList(ListNode head) {
        int count = 0;
        ListNode temp = head;

        // First pass: find the length.
        while (temp != null) {
            count++;
            temp = temp.next;
        }

        // For even length, this selects the second middle.
        int middle = count / 2 + 1;

        // Second pass: move to the middle position.
        temp = head;
        while (temp != null) {
            middle--;

            if (middle == 0) {
                break;
            }

            temp = temp.next;
        }

        return temp;
    }
}

```

Static Visual

Odd length:

```

NULL ← [1] → [2] → [3] → [4] → [5] → NULL
                        ↑
                    middle

```

Even length:

```

NULL ← [1] → [2] → [3] → [4] → [5] → [6] → NULL
                        ↑
                    second middle

```

Complexity

Time: $O(n) + O(n/2) = O(n)$. Space: $O(1)$.

5. Approach 2 — Slow and Fast Pointers

This is the optimized one-pass method. Start both pointers at head. Move slow by one node and fast by two nodes. When fast reaches the end condition, slow is at the required middle. For odd length, fast reaches the last node; for even length, fast reaches null. This naturally gives the second middle for even-length lists.

Pseudocode

```
slow = head
fast = head

while fast != null AND fast.next != null:
    slow = slow.next
    fast = fast.next.next

return slow
```

Original Core Algorithm

```
slow = head
fast = head

while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next

return slow
```

Complete Java Code

```
class Solution {
    public ListNode middleOfLinkedList(ListNode head) {
        // Empty list has no middle node.
        if (head == null) {
            return null;
        }

        // One node is already the middle.
        if (head.next == null) {
            return head;
        }

        ListNode slow = head;
        ListNode fast = head;

        // slow moves 1 step; fast moves 2 steps.
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // Return the actual middle node.
        return slow;
    }
}
```

Static Visual

Odd length:

```
[1] → [2] → [3] → [4] → [5] → NULL
                ↑
            slow
                ↑
            fast (last)
```

Even length:

```
[1] → [2] → [3] → [4] → [5] → [6] → NULL
                ↑
            slow
                fast → NULL
```

6. Why Slow and Fast Works

The fast pointer travels twice as quickly as the slow pointer. Therefore, when fast has covered the full list, slow has covered roughly half of it. This places slow at the middle. The loop condition `fast != null && fast.next != null` safely

handles both odd and even lengths.

Important Java Point

Keep the condition in this order: `fast != null && fast.next != null`. Java evaluates `&&` from left to right, so `fast` must be checked before accessing `fast.next`.

7. Edge Cases

Case	Expected result
<code>head == null</code>	return null
One node	return that node
Two nodes	return the second node
Odd length	return the exact middle
Even length	return the second middle

8. Complexity Comparison

Approach	Time	Extra Space	Passes
Length + traversal	$O(n)$	$O(1)$	2
Slow + fast	$O(n)$	$O(1)$	1

The slow/fast approach is preferred when a one-pass solution is expected. Both approaches use constant auxiliary space.

9. Quick Revision

```

Middle of linked list:
    slow = head
    fast = head

    while fast != null && fast.next != null:
        slow = slow.next
        fast = fast.next.next

    return slow

Key rule:
Odd length  → fast ends at last node
Even length → fast ends at null
Answer      → slow
```

10. Conclusion

The key pattern to remember is the slow-and-fast pointer technique. When both start at head and slow moves one step while fast moves two, slow reaches the middle at the correct stopping condition. For this problem, the code returns the second middle when the list length is even.